SwarRakshak Continuous Full-Stack Development, Audit, and Bug-Fixing Prompt

Role

Act as a senior full-stack engineer responsible for the complete SwarRakshak project. You are an expert in React, JavaScript or TypeScript, Python, FastAPI, SQLite, audio processing, machine learning integration, security, privacy, automated testing, debugging, and software quality assurance.

You are working with a beginner hackathon team. Make safe, understandable, incremental changes. Do not stop after generating code. Continue through implementation, testing, auditing, bug fixing, re-testing, and documentation updates until the project reaches the acceptance criteria or a clearly documented blocker is found.

Product

SwarRakshak is a privacy-aware voice-risk assistant that analyzes call audio and request context to warn users about possible voice-cloning impersonation before a sensitive action is approved.

The system combines:

1. Voice-authenticity evidence.
2. Optional speaker consistency evidence.
3. Caller, urgency, transaction, and procedure context.

It returns a transparent risk score, an understandable risk level, reasons, confidence, and a safe verification recommendation. It does not prove caller identity, automatically block calls, approve payments, reject transactions, or move money.

Required technology stack

Use the original project stack:

- **Frontend:** React with JavaScript or TypeScript.
- **Backend:** Python with FastAPI.

- **Database:** SQLite for the hackathon.
- **Audio processing:** FFmpeg and librosa where appropriate.
- **Machine learning:** PyTorch, torchaudio, scikit-learn, or a suitable pretrained model.
- **Optional speaker comparison:** A suitable speaker-embedding model.
- **Deployment:** A simple local or hosted application; Docker only if already configured or easy to maintain.

Do not migrate to Flutter unless the project owner explicitly requests another technology change.

Main product workflow

Implement and verify this flow:

1. A finance employee starts a new analysis.
2. The user uploads a current-call audio file.
3. The user optionally uploads a consented reference voice sample.
4. The user enters caller and transaction context.
5. The backend validates file type, size, duration, and request values.
6. The backend stores audio temporarily in a private non-public directory.
7. The backend converts and segments the audio.
8. The voice-authenticity module analyzes the segments.
9. The optional speaker-comparison module analyzes the reference sample.
10. The context and policy module converts structured context into signals.
11. The risk engine calculates a score from 0 to 100.
12. The recommendation engine creates a clear next action.
13. The React frontend displays score, level, confidence, reasons, segment results, and recommendation.
14. The backend deletes raw audio by default.
15. The database stores only metadata, reasons, results, and segment information.
16. A fraud analyst may view metadata-only summaries of flagged analyses.

For the hackathon, simulated near-real-time analysis using sequential short segments is acceptable. Do not spend the critical implementation time integrating with telecom carriers,

banking cores, real money movement, or every VoIP platform.

Required risk behavior

Use transparent prototype scoring:

Signal	Maximum contribution
Synthetic-voice indication	40
Speaker mismatch	30
Unknown caller number	10
High-value request	10
Urgent request or procedure bypass	10

Cap the score at 100.

Risk levels:

- Low: 0–29.
- Medium: 30–59.
- High: 60–100.
- Inconclusive: evidence is insufficient, even if the numerical score is low.

Use reason codes such as:

- `SYNTHETIC_AUDIO_PATTERN`
- `REFERENCE_VOICE_MISMATCH`
- `UNKNOWN_CALLER_NUMBER`
- `HIGH_VALUE_REQUEST`
- `URGENT_OR_BYPASS_REQUEST`
- `INSUFFICIENT_AUDIO_EVIDENCE`

A low score must never be described as proof that the caller is genuine. A high score must recommend independent verification.

Required React frontend

Build or maintain these screens:

Home

Explain the problem, privacy approach, and limitation that the result is a warning rather than proof of identity.

Audio input

Provide current-call upload and optional reference-sample upload. Show file name, validation status, duration where available, and a clear raw-audio retention message.

Context form

Collect:

- Caller number status: known, unknown, hidden.
- Request urgency: normal, urgent.
- Transaction value: low, medium, high.
- Procedure status: normal, bypass requested.
- Caller relationship: customer, vendor, executive, unknown.

Validate fields before submitting.

Analysis progress

Show upload, preprocessing, audio analysis, speaker comparison, and scoring states. Prevent duplicate submissions. Handle cancellation and timeout safely where supported.

Result

Show:

- Risk score.
- Risk level.
- Confidence level and score.
- Evidence sufficiency.
- Audio and speaker-analysis summaries.
- Human-readable reason list.

- Segment timeline.
- Recommended action.
- Whether raw audio was deleted.

Do not use color as the only risk indicator. Support keyboard navigation and readable labels.

Analyst summary

Show metadata-only summaries with filtering by risk level and pagination. Never expose raw audio or complete confidential transcripts.

React engineering requirements

- Use a clear `src/` structure such as `components`, `pages`, `services`, `models`, `state`, and `utils`.
- Centralize HTTP calls in a service layer.
- Use typed response models if TypeScript is present; otherwise validate response shapes carefully.
- Use `FormData` for multipart audio uploads.
- Configure the API base URL using environment configuration.
- Never place backend secrets in frontend code.
- Implement loading, empty, validation, unauthorized, forbidden, not-found, server-error, timeout, and unavailable-model states.
- Add tests for validation, model parsing, score display, API error handling, and key user flows.

Required FastAPI backend

Organize the backend into modules for:

- API routes.
- Request and response schemas.
- Audio validation and preprocessing.
- Model adapters.
- Context and policy rules.
- Risk scoring.

- Recommendations.
- Database repositories.
- Privacy cleanup.
- Health checks.

Audio safety requirements

- Validate actual decoded audio, not only file extension.
- Accept WAV, MP3, and M4A where supported.
- Enforce a documented file-size limit, such as 25 MB.
- Enforce a documented duration limit, such as 60 seconds.
- Convert audio to the selected model format.
- Reject silent, corrupted, empty, or unusable files.
- Use secure random temporary filenames.
- Keep temporary audio outside public web directories.
- Delete temporary audio in a `finally` cleanup path on both success and failure.
- Never store raw audio bytes in SQLite.
- Never log raw audio, tokens, passwords, financial credentials, or full transcripts.

If a production-quality model cannot be integrated, use a clearly labeled prototype classifier or prepared demo behavior. Do not claim validated production accuracy.

Required API contract

Implement only these MVP endpoints:

```
GET  /api/v1/health
POST /api/v1/analyses
GET  /api/v1/analyses/{event_id}
GET  /api/v1/analyses
```

`GET /api/v1/health`

Return service status, API version, audio-model availability, speaker-model availability, and

timestamp. Use `200` when usable and `503` when required dependencies are unavailable.

`POST /api/v1/analyses`

Use `multipart/form-data`.

Required fields:

- `current_audio`.
- `caller_number_status`.
- `request_urgency`.
- `transaction_value`.
- `procedure_status`.
- `caller_relationship`.

Optional fields:

- `reference_audio`.
- `retain_audio`, default false.

Return an event ID, lifecycle status, score, level, confidence, evidence sufficiency, audio analysis, speaker analysis, context, reasons, segments, recommendation, privacy status, model version, and rules version.

`GET /api/v1/analyses/{event_id}`

Return one metadata-only analysis. Enforce ownership and analyst/admin permissions. Do not return raw audio.

`GET /api/v1/analyses`

Support metadata-only list results with risk-level filtering, lifecycle-status filtering, ISO date filters, pagination, and safe sorting. Limit `page_size` to 100.

Use consistent JSON errors:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more request fields are invalid.",
    "details": [],
    "request_id": "req_example"
  }
}
```

Handle `400`, `401`, `403`, `404`, `413`, `415`, `422`, `500`, and `503` appropriately.

Required database model

Use SQLite with foreign keys enabled.

`users`

Optional user identity table with `user_id`, `display_name`, optional unique email, role, creation time, and active flag.

`analysis_events`

Central lifecycle table with:

- `event_id` primary key.
- Optional `user_id` foreign key.
- `status`.
- `model_version`.
- `rules_version`.
- `created_at`.
- Optional `completed_at`.
- `raw_audio_retained`.
- Optional `audio_deleted_at`.
- Optional `failure_code`.

`analysis_context`

One-to-one with an event. Store caller-number status, urgency, transaction value, procedure status, caller relationship, optional short safe notes, and creation time.

`analysis_results`

Zero or one per event. Store result ID, event ID, score, risk level, confidence level, optional confidence score, evidence sufficiency, recommended action, and creation time.

`analysis_reasons`

Many per event. Store reason ID, event ID, reason code, message, contribution, source, and display order.

`analysis_segments`

Many per event. Store segment ID, event ID, segment number, start and end seconds, optional synthetic probability, optional confidence score, status, and creation time. Enforce unique `(event_id, segment_number)`.

Required indexes include event creation time, event status, event user, result risk level, reason event ID, reason code, and segment event/order.

Required demo cases

Prepare and verify:

1. Genuine normal call.
2. Synthetic or cloned voice with high-risk context.
3. Natural voice with suspicious context.
4. Inconclusive noisy or invalid audio.

The result screen must explain why each scenario produces its outcome.

Continuous development loop

Follow this loop repeatedly until the acceptance criteria are met:

Loop step 1: Inspect

Inspect the repository tree, package files, environment files, database schema, API routes, frontend screens, tests, and documentation. Determine what is actually implemented. Do not assume the documents and source code match.

Loop step 2: Run baseline checks

Start the backend and frontend. Run health checks. Run the existing test suite. Run formatters, linters, type checks, and static analysis. Capture failures before making changes.

Loop step 3: Compare contracts

Compare PRD, architecture, database schema, API specification, README, frontend requests, backend schemas, and database repositories. Create a mismatch list.

Check especially:

- React versus any stale alternative frontend stack.
- Endpoint paths and HTTP methods.
- Multipart field names.
- Enum values.
- JSON response names and nullable fields.
- Risk thresholds.
- Database column names and foreign keys.
- Privacy claims versus actual deletion behavior.

Loop step 4: Prioritize bugs

Classify each problem:

- Blocker: prevents startup, analysis, or safe operation.
- High: causes incorrect results, data loss, security weakness, or broken API contract.
- Medium: affects important UX, validation, or maintainability.
- Low: cosmetic or non-critical documentation issue.

Fix blockers and high-severity problems first.

Loop step 5: Implement the smallest safe fix

Make focused changes. Do not rewrite unrelated working code. Preserve backward compatibility where practical. Add or update tests with every behavioral fix.

Loop step 6: Run targeted tests

After each fix, run the smallest relevant test first. Then run the complete frontend, backend, API, database, and integration suites.

Loop step 7: Security and privacy review

Verify file validation, path safety, temporary-file cleanup, database exclusion of raw audio, access control, safe logs, CORS, request size limits, and non-disclosure of internal errors.

Loop step 8: Demo review

Run all prepared scenarios through the actual UI and API. Confirm that results are explainable and that high-risk and inconclusive cases produce safe recommendations.

Loop step 9: Documentation review

Update README commands, environment variables, API documentation, database documentation, and known limitations so they match the final code.

Loop step 10: Repeat

Repeat inspection, checks, fixes, and tests until no blocker or high-severity bug remains. Stop only when the project passes the acceptance criteria or a remaining issue requires an external dependency or unavailable credential. Document such blockers clearly.

Deep audit checklist

Frontend

- React starts successfully.
- API base URL is configurable.
- Upload works with valid files.
- Invalid, oversized, empty, noisy, and unsupported files show safe errors.
- Reference audio is optional.
- Duplicate submissions are prevented.

- Loading and timeout states work.
- All documented HTTP errors render safely.
- JSON null and missing fields do not crash the UI.
- Result values are labeled correctly.
- High risk emphasizes independent verification.
- Low risk is not presented as proof of trust.
- Inconclusive results are cautious.
- No secrets or raw audio are logged or stored in browser persistence.
- Responsive and keyboard-accessible behavior is checked.

Backend

- FastAPI starts cleanly.
- Health endpoint reflects actual model availability.
- Request validation matches the API contract.
- File content is validated after decoding.
- Size and duration limits are enforced.
- Temporary files are private and deleted in all paths.
- Score never exceeds 100.
- Every score contribution has a reason.
- Missing speaker reference is handled as unavailable, not mismatch.
- Low evidence is distinct from low risk.
- Database writes are transactional.
- Foreign keys are enabled.
- Unauthorized analysis access is rejected.
- Pagination and filters are validated.
- Errors do not expose stack traces or secrets.

Database

- Fresh schema initializes successfully.
- All six expected tables exist.

- Constraints reject invalid enum values.
- Risk score is restricted to 0–100.
- Probabilities are restricted to 0.0–1.0.
- Event/context and event/result one-to-one constraints work.
- Segment order is unique within an event.
- Indexes exist.
- Raw audio columns do not exist.
- Example records can be inserted and queried.
- Failed transactions roll back cleanly.

Contract and documentation

- README commands match actual files.
- API docs match route behavior.
- Database docs match SQL.
- Product name is consistently SwarRakshak.
- Frontend is consistently documented as React.
- Backend is consistently documented as FastAPI.
- Screenshots and deployment links are either real or clearly marked placeholders.
- Model limitations are stated honestly.

Acceptance criteria

The task is complete only when:

- React frontend runs.
- FastAPI backend runs.
- SQLite schema initializes.
- Health endpoint works.
- Valid analysis request completes.
- Frontend displays score, risk level, reasons, confidence, segments, and recommendation.
- Three core demo scenarios work.
- Inconclusive audio is handled safely.

- Raw audio is deleted by default.
- Raw audio is absent from the database.
- API, database, frontend, and documentation contracts match.
- Automated tests and static checks pass, or remaining failures are explicitly documented.
- No blocker or high-severity bug remains.

Final report required

At the end of every implementation cycle, produce a concise report containing:

1. Repository state inspected.
2. Files changed.
3. Bugs discovered.
4. Severity of each bug.
5. Fix applied for each bug.
6. Tests and commands run.
7. Passed and failed checks.
8. Security and privacy checks completed.
9. Remaining risks or blockers.
10. Exact beginner-friendly run commands.
11. Next loop action if the project is not yet complete.

Do not claim completion without running the checks. Continue the loop until the project is demonstrably working or an external blocker is clearly documented.